App Dev II Project Report

1. Student Details

Name: Ashi Dubey
Roll Number: 24f3003285
Email: 24f3003285@ds.study.iitm.ac.in
I am a Computer Science Engineering student interested in full-stack development and backend systems. This project helped me understand practical backend development using Flask, databases, templates, and role-based workflows.

---

2. Project Details

Project Title: Placement Portal Application (PPA)

Problem Statement: To build a web-based campus recruitment portal that allows institutes (Admin), companies, and students to manage the entire placement process — from company registration and drive creation to student applications and final selections — in a structured, role-based system.

Approach: The application was built using Flask as the REST API backend and Vue 3 as the single-page frontend. The system supports three distinct roles with JWT-based authentication. Celery handles background jobs including daily deadline reminders and monthly placement reports. The frontend communicates entirely through REST APIs with no page reloads.

---

3. AI/LLM Declaration

I used AI to assist in generating boilerplate code and fixing dependency conflicts during development. The extent of AI/LLM usage is approximately 20-25%, primarily for code generation and debugging. All architectural decisions, feature design, testing, and integration were written manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend REST API framework
Flask-SQLAlchemy	ORM for SQLite database interactions
Flask-JWT-Extended	Token-based authentication and role authorization
Flask-Caching	In-memory caching to optimize repeated API calls
Flask-Mail	Sending email notifications and monthly reports
Flask-CORS	Cross-origin resource sharing for Vue frontend
Celery + Redis	Async task queue for CSV export and scheduled jobs
SQLite	Lightweight embedded relational database
Vue 3 (Composition API)	Reactive single-page application frontend
Vue Router	Client-side routing with role-based guards
Axios	HTTP client for API communication
Bootstrap 5	Responsive UI layout and base styling
Chart.js	Admin statistics charts (doughnut, bar, horizontal bar)
python-dotenv	Environment variable management

5. Database Schema / ER Diagram

Tables:

1. User — id, email, password_hash, role (admin/company/student), is_active, is_blacklisted, created_at
2. Student — id, user_id (FK), first_name, last_name, roll_number, branch, year, cgpa, phone, resume_path, skills
3. Company — id, user_id (FK), company_name, hr_contact, hr_email, website, industry, location, approval_status, description
4. Drive — id, company_id (FK), job_title, job_description, eligibility_branch, eligibility_year, min_cgpa, application_deadline, drive_date, location, salary_package, status (Pending/Approved/Closed)
5. Application — id, student_id (FK), drive_id (FK), status (Applied/Shortlisted/Selected/Rejected), application_date, interview_date, interview_location, remarks — with a unique constraint on (student_id, drive_id) to prevent duplicate applications



Relationships:

- User → Student (One-to-One)
- User → Company (One-to-One)
- Company → Drive (One-to-Many)
- Student → Application (One-to-Many)
- Drive → Application (One-to-Many)

The unified User model with a *role* field allows a single login system for all three roles. The unique constraint on Application prevents double-applying at the database level.

---

6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/register	POST	Register student or company
/api/auth/login	POST	Authenticate user, return JWT token

/api/auth/me	GET	Get current user profile
/api/admin/dashboard	GET	Admin KPI stats
/api/admin/companies	GET	List all companies with search/filter
/api/admin/companies/:id/approve	POST	Approve company registration
/api/admin/companies/:id/blacklist	POST	Blacklist a company
/api/admin/drives/:id/approve	POST	Approve a placement drive
/api/admin/students	GET	List all students with search
/api/admin/statistics	GET	Branch/company analytics data
/api/company/profile	GET/PUT	View and update company profile
/api/company/drives	GET/POST	List or create placement drives
/api/company/drives/:id/applications	GET	View applicants for a drive
/api/company/applications/:id/shortlist	POST	Shortlist a student with interview details
/api/company/applications/:id/select	POST	Mark student as selected
/api/company/applications/:id/offer-letter	GET	Download HTML offer letter
/api/student/profile	GET/PUT	View and update student profile
/api/student/resume	POST	Upload resume file
/api/student/dashboard	GET	Eligible drives and application history
/api/student/applications/export	POST	Trigger async CSV export job
/api/student/applications/export/status/:id	GET	Poll Celery task status
/api/student/applications/download/:filename	GET	Download exported CSV
/api/drive/	GET	List approved drives with eligibility filter
/api/drive/:id/apply	POST	Apply to a placement drive

---

7. Architecture and Features

Architecture Overview:

- backend/run.py — Flask application entry point, loads *.env*
- backend/config.py — Centralized configuration (JWT, Redis, Mail, SQLite)
- backend/app/__init__.py — Application factory; registers blueprints, initializes extensions, creates DB tables and seeds admin user
- backend/app/models/ — SQLAlchemy models: User, Student, Company, Drive, Application
- backend/app/routes/ — Flask Blueprints: auth, admin, company, student, drive
- backend/app/tasks/ — Celery tasks: export_tasks, reminder_tasks, report_tasks, celery_config
- backend/app/utils/ — Decorators (role_required, get_current_user_id), file helpers, DB seed
- frontend/src/views/ — Vue SPA pages: Login, Register, AdminDashboard, CompanyDashboard, StudentDashboard
- frontend/src/assets/main.css — Global design system
- frontend/src/api.js — Axios client with JWT interceptor and auto-logout on 401

Implemented Features:

- JWT-based authentication with role-based route guards on both frontend and backend
- Admin can approve/reject companies and drives, blacklist users, view all applications, and see Chart.js analytics
- Companies can create drives (post-approval), manage applications, schedule interviews, shortlist/select/reject students, and generate HTML offer letters

- Students can browse eligible drives (filtered by CGPA, branch, year), apply, track application status, update profile, upload resume, and export history as CSV
- Async CSV export triggered from the student dashboard using Celery; frontend polls for task completion and auto-downloads
- Daily deadline reminders sent via email (Flask-Mail) or Google Chat webhook using Celery Beat (cron: 08:00 UTC)
- Monthly HTML placement report auto-generated and emailed to admin on the 1st of each month (cron: 07:00 UTC)
- In-memory caching (SimpleCache) with configurable Redis upgrade via environment variable
- Duplicate application prevention enforced at both DB constraint and API validation level
- Offer letter generator (optional) produces a formatted HTML document for selected students

---

8. Video Presentation

Drive Link: https://drive.google.com/file/d/1LdPX_dVtGJ58ypzU73QEkoMOl6u1xM0S/view?usp=sharing